

Celebal Technologies Internship – Week 5 Assignment

Apache Spark (PySpark)

Name: Abhishek Raj

Internship: Celebal Technologies Data Engineering Internship

Week: 5

Objective

The objective of this assignment is to understand and implement Apache Spark (PySpark) operations for distributed data processing. The assignment focuses on data cleaning, handling missing values, removing duplicates, filtering records, grouping data, aggregation, timestamp conversion, and building processing pipelines using a custom Retail Sales dataset.

Dataset Description

A custom Retail Sales dataset was created specifically for this assignment. The dataset contains realistic transaction records with multiple stores, regions, cities, product categories, and customer information. It also includes duplicate records, null values, and empty fields to demonstrate real-world data cleaning techniques.

Dataset Columns

- user_id
- transaction_date
- transaction_id
- store_id
- region
- city
- state
- product_category
- product_name

- sale_amount
 - price
 - quantity
 - status
 - age
 - subscription
 - email
 - username
 - raw_timestamp
-

Software Requirements

- Python
 - Apache Spark (PySpark)
 - Java
 - Jupyter Notebook
 - Pandas
 - VS Code
-

Assignment Questions

Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer

Traditional MapReduce writes intermediate results to disk after every processing stage, resulting in high disk I/O and slower execution. It is inefficient for iterative machine learning algorithms, interactive analytics, and real-time applications. Spark overcomes these limitations through in-memory computation, DAG-based execution, optimized APIs, and support for SQL, streaming, machine learning, and graph processing.

Answer

Traditional MapReduce processes data in multiple stages and writes intermediate results to disk after every Map and Reduce operation. This disk-based processing introduces high latency and makes it inefficient for iterative and real-time workloads.

Key limitations of MapReduce include:

- High disk I/O due to writing intermediate results to disk.
- Slow execution for iterative machine learning algorithms.
- Limited support for interactive data analytics.
- Complex programming model requiring separate Map and Reduce functions.
- Not suitable for real-time or streaming applications.

Apache Spark overcomes these limitations by:

- Performing in-memory computation, which significantly improves processing speed.
- Using a Directed Acyclic Graph (DAG) execution engine for optimized task scheduling.
- Providing high-level APIs in Python, Scala, Java, and R.
- Supporting batch processing, SQL, machine learning, graph processing, and stream processing within a single framework.

Q2. Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer

Spark stores intermediate data in memory instead of repeatedly reading and writing to disk. Since machine learning algorithms repeatedly access the same dataset, in-memory caching significantly reduces disk I/O and improves execution speed.

Answer

Apache Spark stores intermediate data in memory (RAM) instead of repeatedly reading from and writing to disk. During iterative algorithms such as machine learning and graph processing, the same dataset is accessed multiple times.

Since Spark keeps frequently used data in memory, it avoids expensive disk I/O operations, resulting in significantly faster execution. Spark also allows datasets to be cached or persisted, enabling efficient reuse across multiple computations.

Compared to traditional disk-based systems like MapReduce, Spark provides much lower latency and better performance for iterative workloads.

Q3. Remove duplicate rows based on user_id and transaction_date.

Explanation

Duplicate records were removed using the dropDuplicates() function by considering the user_id and transaction_date columns. This ensures that only unique transaction records remain.

Code Screenshot

```
duplicate_removed = df.dropDuplicates(
    ["user_id", "transaction_date"]
)

duplicate_removed.show(10)
```

✓ 1.1s

Output Screenshot

user_id	transaction_date	transaction_id	store_id	region	city	state	product_category	product_name	sale_amount	price	quantity	status	age	subscription	email	user
U001	2024-01-20	T00185	S004	East	Philadelphia	PA	Electronics	Mouse	1261.82	1261.82	1	Delivered	44	Premium	user185@gmail.com	u
U001	2024-02-15	T00291	S004	East	Boston	MA	Furniture	Chair	2049.16	512.29	4	Pending	24	Premium	user291@gmail.com	u
U001	2024-02-16	T00433	S005	South	Houston	TX	Books	Spark Handbook	1789.6	447.4	4	Pending	18	Basic	user433@gmail.com	u
U001	2024-02-20	T00243	S004	East	Boston	MA	Books	Spark Handbook	5986.88	1496.72	4	Delivered	25	Basic	user243@gmail.com	u
U001	2024-03-19	T00169	S002	South	Houston	TX	Sports	Football	3521.6	704.32	5	Cancelled	20	Premium	user169@gmail.com	u
U001	2024-04-20	T00383	S005	West	Seattle	WA	Clothing	T-Shirt	100.56	33.52	3	Pending	24	Basic	user383@gmail.com	u
U001	2024-05-30	T00313	S005	South	Dallas	TX	Sports	Football	2787.72	696.93	4	Cancelled	19	Basic	user313@gmail.com	u
U002	2024-04-27	T00369	S002	South	Miami	FL	Furniture	Sofa	1053.4	1053.4	1	Pending	36	Premium	user369@gmail.com	u
U002	2024-05-16	T00317	S005	South	Miami	FL	Sports	Tennis Racket	4071.52	1017.88	4	Delivered	18	Basic	user317@gmail.com	u
U002	2024-06-01	T00052	S005	East	Boston	MA	Furniture	Desk	1571.1	523.7	3	Delivered	20	Premium	user52@gmail.com	u

only showing top 10 rows

Q4. Filter West region and compute average sale_amount.

Explanation

The dataset was filtered to include only records belonging to the West region. The filtered records were grouped by product_category, and the average sale amount was calculated for each category.

Code Screenshot

```
from pyspark.sql.functions import avg

west_sales = (
    df.filter(col("region") == "West")
      .groupBy("product_category")
      .agg(
          avg("sale_amount").alias("Average_Sale")
      )
)

west_sales.show()
```

✓ 0.6s

Output Screenshot

product_category	Average_Sale
Sports	2020.9392
Electronics	1841.69939393938
Clothing	2555.37227272727
Books	2491.0693548387094
Furniture	2019.1579166666663

Q5. Difference between .na.drop() and .na.fill().

Answer

.na.drop() removes rows containing null values, whereas .na.fill() replaces null values with a specified value while retaining all records. In this assignment, missing values in the status column were replaced with "Unknown".

Code Screenshot

```
Answer
```

- `.na.drop()` removes rows containing null values.
- `.na.fill()` replaces null values with a specified value without removing any rows.

Using `.na.fill()` helps preserve the dataset while handling missing values.

```
filled_df = df.na.fill({
    "status": "Unknown"
})

filled_df.select("status").show(10)
```

✓ 0.2s Python

Output Screenshot

status
Pending
Cancelled
Cancelled
Cancelled
Delivered
Unknown
Pending
Delivered
Cancelled
Delivered

only showing top 10 rows

Q6. Count records for each city where count > 100.

Explanation

The dataset was grouped by the city column, and the total number of records was calculated for each city using the count() function. The results were then filtered to display only cities with more than 100 records.

Code Screenshot

```
from pyspark.sql.functions import count

city_count = (
    df.groupBy("city")
      .agg(count("*").alias("Total"))
      .filter(col("Total") > 100)
)

city_count.show()
```

✓ 0.6s

Output Screenshot

```
+----+-----+
|city|Total|
+----+-----+
+----+-----+
```

Q7. Explain the immutability of Spark DataFrames.

Answer

Spark DataFrames are immutable, meaning that once a DataFrame is created, it cannot be modified directly. Any transformation such as dropping columns, renaming columns, or filtering records creates a new DataFrame while leaving the original DataFrame unchanged.

Output Screenshot

Answer

When performing data cleaning operations such as dropping columns, renaming columns, or filtering rows, Spark creates a new DataFrame containing the updated data while leaving the original DataFrame unchanged.

This design ensures:

- Data consistency
- Fault tolerance
- Easy recovery during distributed processing
- Support for parallel execution

```
clean_df = df.drop("state")
clean_df.show(5)
```

✓ 0.2s Python

user_id	transaction_date	transaction_id	store_id	region	city	product_category	product_name	sale_amount	price	quantity	status	age	subscription	email	username
U055	2024-03-12	T00001	S003	East	Philadelphia	Sports	Cricket Bat	1369.66	NULL	2	Pending	22	Premium	user1@gmail.com	user12
U006	2024-03-29	T00002	S004	North	Chicago	Electronics	Keyboard	551.29	551.29	1	Cancelled	21	Basic	user2@gmail.com	user21
U009	2024-04-02	T00003	S001	West	Seattle	Books	Spark Handbook	6652.35	1330.47	5	Cancelled	25	Basic	user3@gmail.com	user31
U027	2024-04-03	T00004	S003	West	Los Angeles	Electronics	Keyboard	2073.15	691.05	3	Cancelled	39	Premium	user4@gmail.com	user41
U088	2024-06-12	T00005	S003	East	Philadelphia	Furniture	Table	2326.36	581.59	4	Delivered	25	Premium	user5@gmail.com	user51

only showing top 5 rows

Q8. Filter users aged 18–30 with Premium subscription.

Explanation

The dataset was filtered using multiple conditions to select users whose age is between 18 and 30 years (inclusive) and whose subscription type is Premium.

Code Screenshot

```
premium_users = df.filter(
    (col("age") >= 18) &
    (col("age") <= 30) &
    (col("subscription") == "Premium")
)

premium_users.show()
```

✓ 0.3s

Output Screenshot

user_id	transaction_date	transaction_id	store_id	region	city	state	product_category	product_name	sale_amount	price	quantity	status	age	subscription	email	usu
U055	2024-03-12	T00001	S003	East	Philadelphia	PA	Sports	Cricket Bat	1369.66	NULL	2	Pending	22	Premium	user1@gmail.com	
U088	2024-06-12	T00005	S003	East	Philadelphia	PA	Furniture	Table	2326.36	581.59	4	Delivered	25	Premium	user5@gmail.com	
U018	2024-05-29	T00007	S005	North	Chicago	IL	Furniture	Desk	3270.4	654.08	5	Pending	20	Premium	user7@gmail.com	
U071	2024-01-17	T00013	S002	North	Detroit	MI	Sports	Cricket Bat	386.9	386.9	1	Cancelled	25	Premium	NULL	
U053	2024-06-17	T00015	S004	South	Dallas	TX	Clothing	T-Shirt	319.2	159.6	2	NULL	19	Premium	user15@gmail.com	
U024	2024-05-17	T00016	S003	West	San Francisco	CA	Books	Python Book	607.1	303.55	2	Pending	25	Premium	user16@gmail.com	
U041	2024-03-07	T00021	S002	West	San Francisco	CA	Sports	Tennis Racket	2441.15	488.23	5	Pending	30	Premium	user21@gmail.com	
U095	2024-03-26	T00028	S002	East	New York	NY	Furniture	Table	226.72	56.68	4	Pending	23	Premium	user28@gmail.com	
U097	2024-06-16	T00037	S001	East	Philadelphia	PA	Clothing	T-Shirt	1524.4	381.1	4	Pending	25	Premium	user37@gmail.com	
U050	2024-01-21	T00040	S002	North	Chicago	IL	Books	SQL Guide	4025.04	1341.68	3	Cancelled	24	Premium	user40@gmail.com	
U007	2024-04-19	T00044	S003	South	Miami	FL	Sports	Tennis Racket	144.24	144.24	1	Pending	28	Premium	user44@gmail.com	
U003	2024-04-17	T00045	S005	South	Houston	TX	Books	SQL Guide	3398.91	1132.97	3	Delivered	29	Premium	user45@gmail.com	
U017	2024-01-06	T00046	S004	West	Seattle	WA	Electronics	Laptop	630.1	315.05	2	NULL	14	Premium	user46@gmail.com	
U002	2024-06-01	T00052	S005	East	Boston	MA	Furniture	Desk	1571.1	523.7	3	Delivered	20	Premium	user52@gmail.com	
U006	2024-01-04	T00056	S001	West	Los Angeles	CA	Books	Python Book	856.71	285.57	3	Cancelled	27	Premium	user56@gmail.com	
U058	2024-01-20	T00060	S005	North	Minneapolis	MN	Electronics	Phone	554.48	277.24	2	Pending	27	Premium	user60@gmail.com	
U085	2024-02-29	T00061	S002	East	Boston	MA	Books	Spark Handbook	501.4	125.35	4	Pending	21	Premium	user61@gmail.com	
U056	2024-04-22	T00063	S003	North	Minneapolis	MN	Clothing	T-Shirt	4095.85	819.17	5	Cancelled	26	Premium	user63@gmail.com	
U085	2024-06-28	T00080	S004	West	Los Angeles	CA	Clothing	Jeans	2612.16	870.72	3	Pending	18	Premium	user80@gmail.com	
U093	2024-06-07	T00081	S002	South	Houston	TX	Furniture	Desk	1924.38	962.19	2	Cancelled	21	Premium	user81@gmail.com	

only showing top 20 rows

Q9. Why handle null values before aggregations?

Answer

Handling null values before performing aggregations ensures accurate analytical results. It prevents incorrect calculations and improves the reliability of statistical measures such as sum, average, minimum, and maximum.

Answer

Handling null values before aggregation helps:

- Improve calculation accuracy.
- Prevent unexpected null results.
- Avoid misleading averages and totals.
- Maintain data quality during analysis.

Replacing or removing null values ensures reliable statistical calculations.

Q10. Cast raw_timestamp and rename it to event_time.

Explanation

The raw_timestamp column was converted to TimestampType and renamed to event_time using the withColumn() function. This conversion enables efficient time-based analysis.

Code Screenshot


```
from pyspark.sql.types import TimestampType

timestamp_df = (
    df.withColumn(
        "event_time",
        col("raw_timestamp").cast(TimestampType())
    )
    .drop("raw_timestamp")
)

timestamp_df.show(5)
```

✓ 0.2s

Output Screenshot

user_id	transaction_date	transaction_id	store_id	region	city	state	product_category	product_name	sale_amount	price	quantity	status	age	subscription	email	user
U055	2024-03-12	T00001	S003	East	Philadelphia	PA	Sports	Cricket Bat	1369.66	NULL	2	Pending	22	Premium	user1@gmail.com	u:
U006	2024-03-29	T00002	S004	North	Chicago	IL	Electronics	Keyboard	551.29	551.29	1	cancelled	21	Basic	user2@gmail.com	u:
U009	2024-04-02	T00003	S001	West	Seattle	WA	Books	Spark Handbook	6652.35	1330.47	5	cancelled	25	Basic	user3@gmail.com	u:
U027	2024-04-03	T00004	S003	West	Los Angeles	CA	Electronics	Keyboard	2073.15	691.05	3	cancelled	39	Premium	user4@gmail.com	u:
U088	2024-06-12	T00005	S003	East	Philadelphia	PA	Furniture	Table	2326.36	581.59	4	delivered	25	Premium	user5@gmail.com	u:

only showing top 5 rows

Q11. Explain Shuffle.

Answer

Shuffle is the process of redistributing data across different partitions during operations such as `groupBy()`, `join()`, and `distinct()`. Since data moves between executors, shuffle is considered a wide transformation and is one of the most resource-intensive operations in Spark.

Answer

Shuffle is the process of redistributing data across different partitions during operations such as:

- `groupBy()`
- `join()`
- `distinct()`
- `reduceByKey()`

Since data moves between executors, network communication is required.

It is called a **wide transformation** because records from one partition may be transferred to multiple other partitions before computation continues.

Q12. Remove rows with null email or empty username.

Explanation

Records containing null values in the email column or empty strings in the username column were removed to improve the quality and consistency of the dataset.

Code Screenshot

```
clean_df = df.filter(  
    col("email").isNotNull() &  
    (col("username") != "")  
)  
  
clean_df.show()  
✓ 0.1s
```

Output Screenshot

user_id	transaction_date	transaction_id	store_id	region	city	state	product_category	product_name	sale_amount	price	quantity	status	age	subscription	email	usu
U055	2024-03-12	T00001	S003	East	Philadelphia	PA	Sports	Cricket Bat	1369.66	NULL	2	Pending	22	Premium	user18@gmail.com	
U006	2024-03-29	T00002	S004	North	Chicago	IL	Electronics	Keyboard	551.29	551.29	1	Cancelled	21	Basic	user2@gmail.com	
U009	2024-04-02	T00003	S001	West	Seattle	WA	Books	Spark Handbook	6652.35	1330.47	5	Cancelled	25	Basic	user3@gmail.com	
U027	2024-04-03	T00004	S003	West	Los Angeles	CA	Electronics	Keyboard	2073.15	691.05	3	Cancelled	39	Premium	user4@gmail.com	
U088	2024-06-12	T00005	S003	East	Philadelphia	PA	Furniture	Table	2326.36	581.59	4	Delivered	25	Premium	user5@gmail.com	
U064	2024-05-25	T00006	S004	North	Detroit	MI	Books	Python Book	2742.88	1371.44	2	NULL	32	Premium	user6@gmail.com	
U018	2024-05-29	T00007	S005	North	Chicago	IL	Furniture	Desk	3270.4	654.08	5	Pending	20	Premium	user7@gmail.com	
U093	2024-04-26	T00009	S003	North	Minneapolis	MN	Books	Python Book	1990.35	663.45	3	Cancelled	42	Premium	user9@gmail.com	
U100	2024-02-09	T00010	S005	West	Seattle	WA	Books	Spark Handbook	4606.4	921.28	5	Delivered	37	Basic	user10@gmail.com	
U094	2024-03-02	T00011	S004	South	Dallas	TX	Electronics	Monitor	1123.17	374.39	3	Delivered	42	Premium	user11@gmail.com	
U070	2024-06-04	T00012	S002	East	Philadelphia	PA	Clothing	Jeans	824.56	412.28	2	Cancelled	27	Basic	user12@gmail.com	
U036	2024-03-25	T00014	S004	West	Seattle	WA	Electronics	Phone	1360.02	1360.02	1	Delivered	35	Premium	user14@gmail.com	
U053	2024-06-17	T00015	S004	South	Dallas	TX	Clothing	T-Shirt	319.2	159.6	2	NULL	19	Premium	user15@gmail.com	
U024	2024-05-17	T00016	S003	West	San Francisco	CA	Books	Python Book	607.1	303.55	2	Pending	25	Premium	user16@gmail.com	
U094	2024-04-26	T00018	S005	West	Los Angeles	CA	Clothing	Jacket	1650.0	412.5	4	Cancelled	40	Basic	user18@gmail.com	
U008	2024-05-18	T00019	S001	East	New York	NY	Books	Python Book	877.19	877.19	1	Cancelled	33	Premium	user19@gmail.com	
U073	2024-06-21	T00020	S002	West	Seattle	WA	Electronics	Phone	900.65	900.65	1	Cancelled	37	Premium	user20@gmail.com	
U041	2024-03-07	T00021	S002	West	San Francisco	CA	Sports	Tennis Racket	2441.15	488.23	5	Pending	30	Premium	user21@gmail.com	
U028	2024-05-24	T00022	S005	North	Detroit	MI	Books	Python Book	698.26	698.26	1	Pending	22	Basic	user22@gmail.com	
U084	2024-05-19	T00023	S005	West	Los Angeles	CA	Books	SQL Guide	1337.02	668.51	2	Delivered	39	Basic	user23@gmail.com	

only showing top 20 rows

Q13. Calculate minimum, maximum, and average price.

Explanation

The .agg() function was used to calculate multiple aggregate statistics on the price column, including the minimum price, maximum price, and average price.

Code Screenshot

```
price_summary = df.agg(
    min("price").alias("Minimum Price"),
    max("price").alias("Maximum Price"),
    avg("price").alias("Average Price")
)

price_summary.show()
```

✓ 0.2s

Output Screenshot

```
+-----+-----+-----+
|Minimum Price|Maximum Price|   Average Price|
+-----+-----+-----+
|      24.05|      1499.37|761.7716833667345|
+-----+-----+-----+
```

Q14. Risk of using inferSchema=True with inconsistent dates.

Answer

Using `inferSchema=True` on datasets containing inconsistent date formats may lead to incorrect data type inference. Dates may be interpreted as strings or converted to null values, affecting filtering, aggregation, and analytical accuracy. Defining an explicit schema is generally recommended for production datasets.

Answer

Using `inferSchema=True` with inconsistent date formats may lead to:

- Incorrect data type inference.
- Dates being interpreted as strings.
- Invalid values becoming null.
- Inaccurate filtering and aggregations.
- Additional preprocessing before analysis.

For production datasets, defining an explicit schema is often recommended.

Q15. Final Processing Pipeline

Explanation

A complete data processing pipeline was created by removing duplicate records, replacing null values in the price column with 0, grouping the dataset by store_id, and calculating the total revenue for each store.

Code Screenshot

```
from pyspark.sql.functions import sum, round

final_pipeline = (
    df.dropDuplicates()
      .na.fill({"price": 0})
      .groupBy("store_id")
      .agg(
        round(
          sum("price"),
          2
        ).alias("Total Revenue")
      )
)

final_pipeline.show()
```

✓ 0.6s

Output Screenshot

```
+-----+-----+
|store_id|Total Revenue|
+-----+-----+
|  S004 |    76194.44 |
|  S001 |    51383.36 |
|  S002 |    81721.86 |
|  S005 |    76957.89 |
|  S003 |    80543.45 |
+-----+-----+
```

Conclusion

This assignment demonstrated the practical implementation of Apache Spark (PySpark) for distributed data processing and data engineering tasks. Various operations such as duplicate removal, missing value handling, filtering, grouping, aggregation, timestamp conversion, and pipeline creation were successfully performed on a custom Retail Sales dataset. The project highlights the efficiency of Spark's in-memory processing and its capability to handle large-scale datasets effectively.

References

- [Apache Spark Documentation](#)
- [PySpark API Documentation](#)
- Celebal Technologies Data Engineering Internship Material